Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

May 2, 2020 · 7 min read · ✦ · Listen

Save

HTTPS://TOWARDSDATASCIENCE.COM/TAGGED/SPACE-SCIENCE-WITH-PYTHON

Space Science with Python — The dance of Venus

Part 4 of the tutorial series takes a look how to determine a perfect timing for a great Venus & Moon photo

Photo by Grant McIver on Unsplash

Preface

This is the 4th part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!

Introduction

Did you recently take a look at the sky in the evening? Right after sunset? You may have recognised this very bright star in the sky that is visible for a few hours before it vanishes below the horizon. In a few days, this dot will be gone… and it will reappear in the morning hours right before sunrise. The *evening star* will become the *morning star*.

This is not a star, but our neighbour planet Venus. In the photo above you can see Venus right below the crescent Moon. Together with Australia's iconic Uluru this picture is a nice shot of our planet's landscape with our cosmic neighbours.

But when are the Moon and Venus is this close constellation? In the last couple of weeks Venus was clearly visible so how many opportunities did we have to see (and to photograph) this beautiful constellation? Python + SPICE can answer that!

We will reduce the underlying complexity of this problem to a few simple computations that will give us an easy answer to our question (the ***Conclusion & Outlook*** *section at the end will list additional computational steps that are needed to get an entire expert's view).*

Let's start

What do we need to determine the points of time where Venus and Moon are close to each other? Well, we need to compute the phase angle between the Moon and Venus as seen from our home planet. The phase angle is the angle that is enclosed by the two Earth-centric directional vectors that point to the Moon and to Venus.

We will compute the angular distance between both objects for a certain time period. During the time both objects move along the Sky with changing angular

distances to the Sun. We need also to compute a minimum distance between Sun — Venus and Sun — Moon. Otherwise we might compute constellations where all three objects are too close (Please note: Never look directly into the Sun; not with your naked eye and not through any devices like a camera's viewfinder. Wait after sunset to observe and photograph both objects safely. Trying to find Venus or other objects close to the Sun may cause blindness and / or irreparable damages to your eyes!).

For this tutorial part our SPICE kernel meta file contains only two kernel paths. *de432s.bsp* is required for the position computation of the celestial bodies and *naif0012.tls* is the kernel that contains temporal information like leap-seconds.

```
\begindata

KERNELS_TO_LOAD = (
 '../_kernels/spk/de432s.bsp',
 '../_kernels/lsk/naif0012.tls',
 )
```

After loading the meta file with *furnsh*, we can define a time interval for our computations: the initial time is set to the beginning of the year 2020 and the end date is the 1st June 2020. The created date-time strings are converted to Ephemeris time (ET) using the SPICE function *utc2et*. The array *TIME_INTERVAL* contains all ETs from the initial time to the end time, in 3600 seconds (1 hour) steps.

To store all relevant information in a common place we use a *pandas dataframe* (line 3). The ET array is stored (line 6) as well as the corresponding UTC time stamps that are being computed in line 12 +13, by applying the *spiceypy* function *et2datetime*. In line 22 to 28 we compute the phase angle between Venus and the Sun as seen from the Earth. For this purpose, we use the SPICE function *phaseq*. The function returns the angular distance in radians between two objects at a certain Ephemeris time *et*. The documentation provides a schematic overview on the input parameters *target* (target body), *illmn* (illuminating body), and *obsrvr* (observer body):

```
                  ILLMN      OBS
ILLMN as seen       ^        /
from TARG at        |       /
ET - LT.            |      /
                  >|..../< phase angle
                    |   /
                  . |  /
                .   | /
               .    |v     TARG as seen from OBS
        SEP   .   TARG    at ET
               .  /
                 /
                v
```

Applying the function and the computational logic of *phaseq* on our problem leads to the following input parameters:

- *target*: The Earth (NAIF ID 399)
- *illmn*: The Sun (NAIF ID 10)
- *obsrvr*: Venus (NAIF ID 299)

Further, we may notice another input parameter called *abcorr*. In our last tutorials, we computed state and directional vectors, phase angles and other parameters in rather simple “steady states”. Here, we compute the distances in a more precise way, by considering the light time (travelling time of the light) and taking the observes velocity into account. The resulting input argument is *LT+S*.

The corresponding SPICE documentation for corrections provides the following descriptions for the terms

LT

> *Correct for one-way light time (also called "planetary aberration") using a Newtonian formulation. This correction yields the state of the target at the moment it emitted photons arriving at the observer at ET.*

LT+S

> *Correct for one-way light time and stellar aberration using a Newtonian formulation. This option modifies the state obtained with the 'LT' option to account for the observer's velocity relative to the solar system barycenter. The result is the apparent state of the target — -the position and velocity of the target as seen by the observer.*

These corrections are necessary in our following tutorials, where we will also compute the positions of hazardous Asteroids where a correct timing and observation planning are crucial.

Part 2/7

Now we use the same function to compute the phase angle between the Moon and the Sun…

Part 3/7

… as well as the phase angle between the Moon and Venus.

Part 4/7

We can now set our requirements for a photogenic Moon — Venus constellation. A new column is added called *PHOTOGENIC*, where a binary value represents a “nice” (1) or “not so nice” (0) constellation. Please note that we set some artificial requirements for this computation. Tastes are different in (Astro-) Photography; so feel free to change the requirements accordingly.

Here we apply the following requirements in an if-else condition within the lambda function:

- The angular distance between the Sun and Venus shall be larger than 30°
- The angular distance between the Sun and the Moon shall be larger than 30°
- The angular distance between the Moon and Venus shall be smaller than 10°

Part 5/7

Let's have a look at the results. We computed the phase angles in a time interval of 3649 hours (152 days). Only 143 hours (6 days) fulfil this requirement!

Part 6/7

We can now show the results in a plot to get a feeling of the changing angular distances and the perfect time slots for our observations. For this purpose, we use *matplotlib*. Line 10 to 17 plot the three different phase angles vs. the dates. We set some labels for the axes, create a vertical grid for better readability and set the limits for the x axis (line 20 to 27). For the x axis we format the design and locators as little bit (line 30 to 34). Only the beginning of a month is displayed and each

smaller locator represents an individual day of the month. The “photogenic moments” are visualized as vertical blue lines and are created in a for-loop in line 38+39, using the function *axvline*. To extract the “photogenic moments” dates, a filtering is applied using the *pandas* function *.loc[]*:

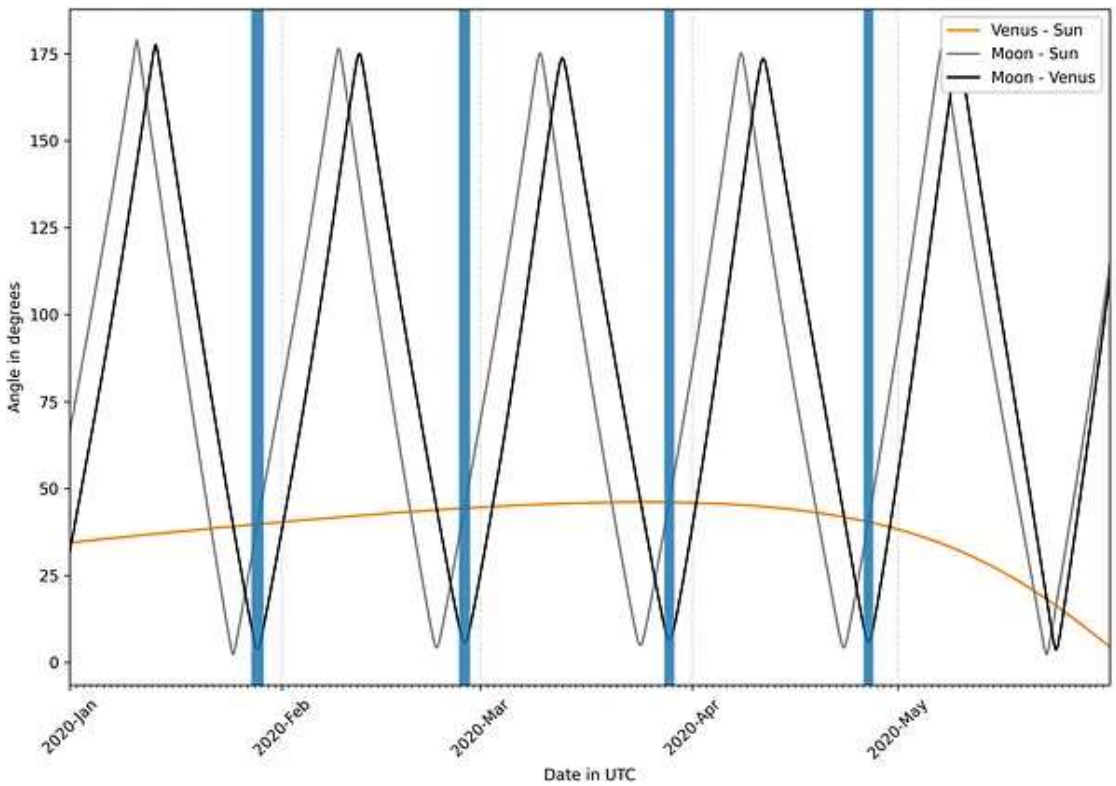
```
INNER_SOLSYS_DF.loc[INNER_SOLSYS_DF[‘PHOTOGENIC’] == 1][‘UTC’]
```

Finally, we add the legend, rotate the dates slightly and store the plot.

Part 7/7

The figure below shows the resulting plot. As you can see, the angles are displayed vs. the date. One Moon orbit around the Earth is around 27 days, resulting in the fast changing pattern for the phase angles Moon — Sun and Moon — Venus, respectively. The angular distance between Venus and the Sun reached its maximum a few weeks ago and currently the distance is rapidly decreasing. End of May, the Venus will no be visible since Venus is between the Sun and Earth. This position constellation is called “lower conjunction”. In a few weeks, Venus will re-appear as the morning star.

The blue lines indicate the “photogenic moments” we have defined earlier. Each Moon revolution contains only 1 to 1.5 days of “the perfect moment”. Did you take a photo in one of these time slots?



Phase angles vs. the date. The orange, grey and black curves show the phase angles of Venus — Sun, Moon — Sun and Moon — Venus, respectively. The blue vertical lines indicate the “photogenic moments”, where certain angular requirements are met that we defined earlier. Credit: T. Albin

Conclusion & Outlook

Unfortunately, there is no opportunity left for May; so we have to wait for the morning star (and motivate ourselves to wake up early). Or we wait … for the evening star. But when is Venus returning in the evening? With the SPICE functions you learned so far you can try to answer this question! Try to plot the orbit of Venus and Earth as well as the phase angle between Venus and Sun to get an idea of the returning dates.

If you are an experienced astronomer you may point out that we did not compute the position of Venus on the sky. Further, we computed the phase angles from the centre of the Earth, not considering the geographical coordinates of a human observer. Well, you are absolutely right!

An overlapped simulation of the Moon's sky position depending on three observer locations (all three locations have the same geographical latitude as the German city Stuttgart). Additionally to Stuttgart you can see the position of the Moon as seen from the Equator and at 70° south. The date-time is 2020–03–28 at 20:00:00 UTC. You can see nicely the close constellation with Venus, the Pleiades and Aldebaran. The three merged sub-figures have been created with the planetarium software Stellarium

So far, we learned some SPICE functions and some use cases using Python and Jupyter Notebooks. In the next tutorials I would like to dive deeper, beginning with asteroids and so-called Near-Earth objects (NEOs). But before, however, we need to cover two general topics:

- Reference Frames
- Orbital Mechanics

These two topics are as relevant for us as mathematics and statistics for Data Science and Machine Learning. Do not worry, we will learn these things together, with Python. Afterwards you have a solid knowledge fundament to work on science topics I will show you afterwards.

In the meantime, play around with the functions you have learned so far. Improve the *matplotlib* plots or create interactive versions using *bokeh*.

Thomas

Space Science Python Programming Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play